

Homework 7: Undecidability

Will Griffin

April 9, 2026

1. Bounds checking

- (a) Prove that it is undecidable whether a TM M , on input w , ever attempts to move its head past the left end of the tape.

Let LEFT_{TM} be the language where

$$\text{LEFT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ attempts to move its head past the left end of the tape}\}$$

Assume LEFT_{TM} is decidable by a TM R .

Construct a TM S that decides A_{TM} .

$S =$ “On input $\langle M, w \rangle$,

1. Construct a TM $M' =$ “On input x :
 - a. Simulate M on $\#w$. If the simulation of M ever tries to move to the cell containing $\#$, move the head to the right and continue the simulation.
 - b. If M halts, attempt to move the head of M' left past the first cell.”
2. Run R on $\langle M', \varepsilon \rangle$.
3. If R accepts, accept.
4. If R rejects, reject.”

On a case basis:

- M accepts w : M' will move left past the first cell. R accepts $\langle M', \varepsilon \rangle$. S accepts $\langle M, w \rangle$.
- M rejects w : M' will move left past the first cell. R accepts $\langle M', \varepsilon \rangle$. S accepts $\langle M, w \rangle$.
- M loops on w : M' will not move left past the first cell. R rejects $\langle M', \varepsilon \rangle$. S rejects $\langle M, w \rangle$.

Since S decides A_{TM} , but A_{TM} is undecidable, we have a contradiction. Therefore, LEFT_{TM} is undecidable.

- (b) Prove that it is decidable whether a TM M , on input w , ever attempts to move its head past the right end of the input string w . Your answer should be a high-level description of a TM.

Construct a TM R that decides whether a TM M , on input w , attempts to move its head past the right end of w . This is done by simulating M on w . The total number of configurations M can be in, C , is equal to $C = |Q| \times |w| \times |\Gamma|^{|w|}$. Simulate M on w for at most C steps.

Between each step in the situation, if the head moves right past the position n , *accept*. If M halts without ever moving right past the position n , *reject*. After C steps of simulation, if M has not halted, *reject*.

2. The Power of 10

Know:

- A 10-compliant TM always halts on every input
- It is decidable whether a TM is 10-compliant

Consider the language

$$L_2 = \{\langle M \rangle \mid M \text{ is a 10-compliant TM that rejects } \langle M \rangle\}$$

- (a) Prove that L_2 is decidable.

If it is decidable whether a TM M is 10-compliant, then there is some decider R that decides 10-compliance for a TM M .

To prove that L_2 is decidable, construct a TM S that operates on some input w . If $w \neq \langle M \rangle$ (w is not a valid TM encoding), *reject*. Run the decider R on $\langle M \rangle$. If M is not 10-compliant, *reject*. If M is 10-compliant, simulate M on $\langle M \rangle$. Since M is 10-compliant, this simulation will eventually halt. If M rejects $\langle M \rangle$, *accept*, otherwise, if M accepts $\langle M \rangle$, *reject*.

Since this TM S halts on any input and only accepts TMs that are both 10-compliant and reject their own encoding, L_2 is decidable.

- (b) Prove that any TM that decides L_2 must not be 10-compliant.

Any TM whose encoding is in L_2 must reject its own encoding. However, S , the decider for L_2 , accepts if the TM it operates on rejects itself. Therefore, for S to be 10-compliant, it must reject itself, but by determining whether S is in L_2 , if S rejects $\langle S \rangle$, then S checking $\langle S \rangle$ accepts. This causes a paradox.

By this contradiction, any decider for L_2 is not 10-compliant.

- (c) Where in your solution to (a) is the violation of 10-compliance? (Full credit for any reasonable answer; I just want you to think about it).

Since deciding L_2 is a non-10-compliant operation, any machine that decides L_2 is not 10-compliant. Therefore, in running S on $\langle S \rangle$, the 10-compliance-decider step in S breaks 10-compliance for S .

3. Grep and Sed

- (a) Prove that it is decidable whether 2 regular expressions are equivalent to each other. Hint: prove that it is decidable whether a DFA recognizes the empty language. Then, use closure properties.

For two regular expressions α_1 and α_2 that recognize the languages L_1 and L_2 respectively, if α_1 and α_2 are equivalent, then $L_1 \cap L_2 = L_1 \cup L_2$. The difference between these two languages ($L_{\text{diff}} = (L_1 \cap \overline{L_2}) \cup (L_2 \cap \overline{L_1})$) would then be the empty language $L_{\text{diff}} = \emptyset$.

Therefore, since the closure operations of complementation, union, and intersection are closed under regular languages, it is enough to prove whether determining a DFA recognizes the empty language L_{diff} is decidable.

To do this, construct some TM $M_\emptyset$ that operates on the input string of $\langle M_{\text{diff}} \rangle$ (the DFA constructed from L_{diff}). Search for a path to any accept state in M_{diff} . If any accept state is reachable, *reject*. If no accept state is reachable, *accept*.

Since a TM that halts on any input of $\langle M_{\text{diff}} \rangle$ can be constructed to decide L_{diff} , L_{diff} is decidable. By closure properties of complementation, union, and intersection under regular languages, this can be extended to say that it is decidable whether 2 regular expressions are equivalent.

- (b) Prove that it is undecidable whether two **sed** programs (as defined in CP3) are equivalent to each other. Hint: use CP3 part 3

In CP3 part 3, we created a program that can convert any Turing machine into an equivalent **sed** script. By this implementation, this proves **sed** to be **Turing-complete**. Turing-completeness means that the model of computation is at least equal in power to Turing machines. However, since Turing machines are the most powerful method of computation, **sed** is equal in computational power to Turing machines. Therefore, we can say that for two **sed** programs S_1 and S_2 , we can construct two equivalent TMs M_1 and M_2 respectively. However, by Theorems 5.4 and 5.2 (Sipser), it is undecidable whether two TMs are equivalent. Therefore, because **sed** is Turing-complete, it is also undecidable whether two **sed** programs are equivalent to each other.